Lista Dwukierunkowa

1

Generated by Doxygen 1.18.0

1 Directory Hierarchy 1

1.1 Directories . . . 1

2 Class Index 3

2.1 Class List . . . 3

3 File Index 5

3.1 File List . . . 5

4 Directory Documentation 7

4.1 docs Directory Reference . . . 7

4.1.1 Detailed Description . . . 7

5 Class Documentation 9

5.1 LinkedList Class Reference . . . 9

5.1.1 Detailed Description . . . 10

5.1.2 Member Function Documentation . . . 10

5.1.2.1 addAtIndex() . . . 10

5.1.2.2 addBack() . . . 10

5.1.2.3 addFront() . . . 10

5.1.2.4 removeAtIndex() . . . 11

5.2 ListIterator Class Reference . . . 11

5.3 Node Class Reference . . . 11

5.4 NodeFactory Class Reference . . . 11

6 File Documentation 13

6.1 NodeFactory.h . . . 13

6.2 Iterator.h . . . 13

6.3 src/List/LinkedList.h File Reference . . . 14

6.3.1 Detailed Description . . . 14

6.4 LinkedList.h . . . 14

6.5 Node.h . . . 15

Index 17

Chapter 1

Directory Hierarchy

1.1 Directories

docs 7

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

LinkedList
: Klasa implementująca dynamiczną listę dwukierunkową w języku C++ 9
ListIterator 11
Node 11
NodeFactory 11

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

src/Factory/NodeFactory.h 13
src/Iterator/Iterator.h 13
src/List/LinkedList.h
Definicja klasy reprezentującej dwukierunkową listę wiązaną 14
src/List/Node.h 15

Chapter 4

Directory Documentation

4.1 docs Directory Reference

4.1.1 Detailed Description

Implementacja listy dwukierunkowej w C++

Chapter 5

Class Documentation

5.1 LinkedList Class Reference

Klasa implementująca dynamiczną listę dwukierunkową w języku C++.

`#include <LinkedList.h>`

Public Member Functions

- **LinkedList** ()

  Konstruktor domyślny. Tworzy pustą listę.
- ~**LinkedList** ()

  Destruktor. Zwalnia pamięć zajmowaną przez wszystkie węzły listy.
- void addFront (int value)

  Dodaje nowy element na początek listy.
- void addBack (int value)

  Dodaje nowy element na koniec listy.
- void addAtIndex (int index, int value)

  Wstawia nowy element pod wskazany indeks. @indeks Indeks, pod który chcemy wstawić element (liczony od 0).
- void **removeFront** ()

  Usuwa pierwszy element z listy.
- void **removeBack** ()

  Usuwa ostatni element z listy.
- void removeAtIndex (int index)

  Usuwa element znajdujący się pod wskazanym indeksem.
- void **displayForward** () const

  Wyświetla zawartość listy od początku do końca (w przód).
- void **displayBackward** () const

  Wyświetla zawartość listy od końca do początku (w tył).
- void **clear** ()

  Usuwa wszystkie elementy z listy i resetuje jej stan.

5.1.1 Detailed Description

Klasa implementująca dynamiczną listę dwukierunkową w języku C++.

Klasa zarządza węzłami typu Node, umożliwiając efektywne operacje dodawania, usuwania oraz przeglądania elementów zarówno w przód, jak i w tył.

5.1.2 Member Function Documentation

5.1.2.1 addAtIndex()

```
void LinkedList::addAtIndex (
            int index,
            int value)
```

Wstawia nowy element pod wskazany indeks. @indeks Indeks, pod który chcemy wstawić element (liczony od 0).

Parameters

value	Wartość typu całkowitego do wstawienia.

5.1.2.2 addBack()

```
void LinkedList::addBack (
            int value)
```

Dodaje nowy element na koniec listy.

Parameters

value	Wartość typu całkowitego do dodania.

5.1.2.3 addFront()

```
void LinkedList::addFront (
            int value)
```

Dodaje nowy element na początek listy.

Parameters

value	Wartość typu całkowitego do dodania.

5.1.2.4 removeAtIndex()

```
void LinkedList::removeAtIndex (
            int index)
```

Usuwa element znajdujący się pod wskazanym indeksem.

Parameters

index	Indeks usuwanego elementu (liczony od 0).

The documentation for this class was generated from the following files:

- src/List/LinkedList.h
- src/List/LinkedList.cpp

5.2 ListIterator Class Reference

Public Member Functions

- **ListIterator** (Node *ptr)
- int & **operator*** () const
- ListIterator & **operator++** ()
- ListIterator & **operator--** ()
- bool **operator==** (const ListIterator &other) const
- bool **operator!=** (const ListIterator &other) const
- Node * **getNode** () const

The documentation for this class was generated from the following file:

- src/Iterator/Iterator.h

5.3 Node Class Reference

Public Member Functions

- **Node** (int val)

Public Attributes

- int **data**
- Node * **prev**
- Node * **next**

The documentation for this class was generated from the following file:

- src/List/Node.h

5.4 NodeFactory Class Reference

Static Public Member Functions

- static Node * **createNode** (int value)

The documentation for this class was generated from the following file:

- src/Factory/NodeFactory.h

Chapter 6

File Documentation

6.1 NodeFactory.h

```
00001 #ifndef NODE_FACTORY_H
00002 #ifndef NODE_FACTORY_H
00003 #define NODE_FACTORY_H
00004
00005 #include "./List/Node.h"
00006
00007 class NodeFactory {
00008 public:
00009     // Metoda fabrykująca (statyczna), która tworzy i zwraca nowy węzeł
00010     static Node* createNode(int value) {
00011         return new Node(value);
00012     }
00013 };
00014
00015 #endif
00016 #endif
```

6.2 Iterator.h

```
00001 #ifndef ITERATOR_H
00002 #define ITERATOR_H
00003
00004 #include "./List/Node.h"
00005
00006 class ListIterator {
00007 private:
00008     Node* current; // Wskaźnik na aktualny węzeł w iteracji
00009
00010 public:
00011     // Konstruktor
00012     explicit ListIterator(Node* ptr) : current(ptr) {}
00013
00014     // Przeciążenie operatora wyłuskania (*it), aby zwracał wartość węzła
00015     int& operator*() const {
00016         return current->data;
00017     }
00018
00019     // Przeciążenie operatora inkrementacji prefixowej (++it) - przejście do następnego elementu
00020     ListIterator& operator++() {
00021         if (current != nullptr) {
00022             current = current->next;
00023         }
00024         return *this;
00025     }
00026
00027     // Przeciążenie operatora dekrementacji prefixowej (--it) - przejście do poprzedniego elementu
00028     ListIterator& operator--() {
00029         if (current != nullptr) {
00030             current = current->prev;
00031         }
00032         return *this;
00033     }
00034
```

```
00035     // Sprawdzenie równości (czy iteratory wskazują na ten sam element)
00036     bool operator==(const ListIterator& other) const {
00037         return current == other.current;
00038     }
00039
00040     // Sprawdzenie nierówności (!=)
00041     bool operator!=(const ListIterator& other) const {
00042         return current != other.current;
00043     }
00044
00045     // Metoda pomocnicza zwracająca surowy wskaźnik (jeśli potrzebny)
00046     Node* getNode() const {
00047         return current;
00048     }
00049 };
00050
00051 #endif
```

6.3 src/List/LinkedList.h File Reference

Definicja klasy reprezentującej dwukierunkową listę wiązaną.

```
#include "Node.h"
```

Classes

- class LinkedList

  Klasa implementująca dynamiczną listę dwukierunkową w języku C++.

6.3.1 Detailed Description

Definicja klasy reprezentującej dwukierunkową listę wiązaną.

6.4 LinkedList.h

Go to the documentation of this file.

```
00001 #ifndef LINKED_LIST_H
00002 #define LINKED_LIST_H
00003
00004 #include "Node.h"
00005
00010
00018 class LinkedList {
00019 private:
00020     Node* head;
00021     Node* tail;
00022     int size;
00023
00024 public:
00028     LinkedList();
00029
00033     ~LinkedList();
00034
00039     void addFront(int value);
00040
00045     void addBack(int value);
00046
00052     void addAtIndex(int index, int value);
00053
00057     void removeFront();
00058
00062     void removeBack();
00063
```

```
00068     void removeAtIndex(int index);
00069
00073     void displayForward() const;
00074
00078     void displayBackward() const;
00079
00083     void clear();
00084 };
00085
00086 #endif
```

6.5 Node.h

```
00001 #ifndef NODE_H
00002 #define NODE_H
00003
00004 class Node {
00005 public:
00006     int data; // Wartość przechowywana w węźle
00007
00008     Node* prev; // Wskaźnik na poprzedni węzeł
00009
00010     Node* next; // Wskaźnik na następny węzeł
00011
00012     Node(int val) : data(val), prev(nullptr), next(nullptr) {}
00013 };
00014
00015 #endif
```

Index

addAtIndex
    LinkedList, 10
addBack
    LinkedList, 10
addFront
    LinkedList, 10

docs Directory Reference, 7

LinkedList, 9
    addAtIndex, 10
    addBack, 10
    addFront, 10
    removeAtIndex, 10
ListIterator, 11

Node, 11
NodeFactory, 11

removeAtIndex
    LinkedList, 10

src/Factory/NodeFactory.h, 13
src/Iterator/Iterator.h, 13
src/List/LinkedList.h, 14
src/List/Node.h, 15